

APPENDIX

In the appendix, we provide additional material that complements the main results, including a finer-grained taxonomy of defect-detection and code-improvement comments, as well as a cross-agent analysis.

A. Fine-Grained Technical Taxonomy of Defect Detection and Code Improvement

In RQ1 III-A, we use Bacchelli and Bird’s taxonomy [9] to compare the similarities and differences in review comments from human and AI agent reviewers. To complement this high-level comparison, we apply the finer-grained taxonomy of Turzo and Bosu [40] to examine Defect Detection and Code Improvement comments, as shown in Table A1.

TABLE A1: Taxonomy of Defects and Code Improvements, adapted from Turzo and Bosu [40].

	Category	Description
Defect Detection	Functional defect	Functionality is missing or incorrect and often requires additional code or modifications.
	Logical	Control flow, comparison-related, and logical errors.
	Validation	Validation mistakes or mistakes made when detecting an invalid value are of this class.
	Resource	Resource (variables, memory, files, database) initialization, manipulation, and release.
	Timing	Potential issues due to incorrect thread synchronization.
	Support	Issues related to support systems and libraries or their configurations.
Code Improvement	Interface	Mistakes when interacting with other parts of the software, such as an existing database or operating system.
	Solution approach	Suggestions to adopt an alternate algorithm or data structure.
	Alternate Output	Suggestions to modify an error or toast message, or to change the value returned by a function.
	Code Organization	Refactoring suggestions.
	Convention	Violations of identifier naming conventions.
	Visual Representation	Whitespace, blank lines, code rearrangements, and indentation-related comments.
	Documentation	Suggestions to add or modify comments or documentation to aid code comprehension.

B. Cross-Agent Analysis

To examine whether individual AI agent reviewers differ in their review behavior, we conduct a cross-agent analysis of the seven agents identified in Section II-F, including CodeRabbit [5], GitHub Copilot [6], Gemini Code Assist [31], Cursor [32], Claude Code [33], Sourcery [34], and Greptile [35]. For each RQ, we randomly sample 383 observations for each of the seven AI-agent reviewers. We calculate the same metrics reported in Section III for every agent. We then compare each agent’s metric values with the corresponding aggregate values for all AI-agent reviewers.

B.1 Review comment characteristics in RQ1.

To understand whether the characteristics of review comments vary across AI-agent reviewers, we analyze feedback-type distributions, reading-ease scores, and quality-attribute distributions separately for each of the 7 agents. We then compare the results of each agent with the aggregate results of all AI-agents.

Feedback type. We examine whether the distribution of feedback types differs across AI-agent reviewers. As shown in Figure A1, Cursor’s feedback-type distribution differs significantly from the aggregate distribution for all AI-agent reviewers, with a large effect size. Among Cursor comments, 99% focus on Defect Detection, compared with 37% of AI-agent comments overall.

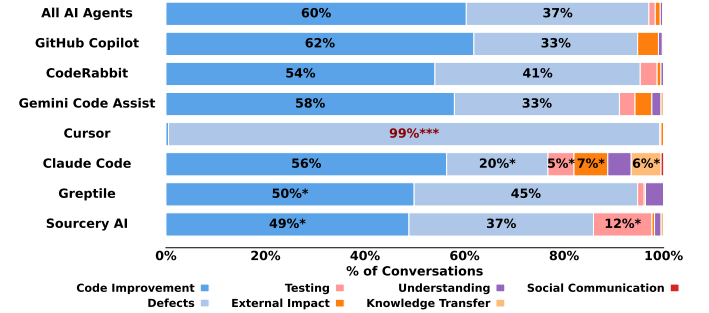


Fig. A1: Feedback type distribution for each AI review agent, compared against the aggregate “All AI Agents” baseline reported in RQ1. */**/* indicates a significant difference from the baseline with a small/medium/large effect size.

Reading ease. We then rank the reading ease scores of the 7 agents and compare each agent with the aggregate AI-agent baseline using the Scott-Knott ESD test. As shown in the Figure A2, review comments generated by Gemini Code Assist have a significantly higher reading ease score than the overall AI agent baseline, with a mean score of 48.7 compared with 43.0 overall. In contrast, comments generated by Sourcery AI and Cursor have significantly lower reading ease scores. The remaining AI agent reviewers (i.e., GitHub Copilot, Greptile, Claude Code, and CodeRabbit) show no significant difference in reading ease with the overall AI agent baseline.

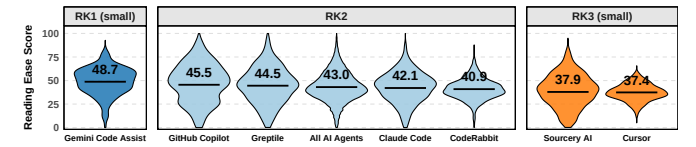


Fig. A2: Scott-Knott ESD ranking of Flesch-Kincaid Reading Ease for each agent and the “All AI Agents” baseline.

Quality attributes. Finally, we compare the distribution of comment quality attributes [37] for each AI agent reviewer with the aggregate AI-agent baseline using a chi-square test [39]. As shown in Table A2, 86.4% of comments generated by Cursor identify a code weakness (*Evaluation*) without providing a suggestion for improvement, compared with 4.6% of comments generated by AI-agent reviewers overall. By contrast, 13.3% of comments generated by Cursor both identify a code weakness and provide a suggestion

for improvement, compared with 91.1% overall. Comments generated by Greptile are significantly less likely (medium effect size) to both identify a code weakness (*Evaluation*) and provide a suggestion for improvement, at 71.3% versus 91.1% overall. Claude Code shows statistically significant differences from the aggregate quality-attribute distribution with small effect sizes, whereas GitHub Copilot, CodeRabbit, and Gemini Code Assist show no significant differences.

TABLE A2: Distribution of quality attributes among AI agent reviewers. M, Q, E, and S denote Emotion, Question, Evaluation, and Suggestion, respectively. **Green** marks an attribute present and **Red** marks one absent. “—” indicates fewer than 1% of comments. “All Agents” denotes the aggregate AI agent.

Agent	MQES	MQES	MQES	MQES	MQES	Total
All Agents	2.9%	—	91.1%	4.6%	1.2%	100%
Copilot	1.3%	—	93.2%	2.6%	2.9%	100%
CodeRabbit	3.9%	—	95.3%	—	—	100%
Gemini	3.7%	—	91.6%	1.6%	1.3%	100%
Cursor	—	—	13.3%***	86.4%***	—	100%
Claude	7.3%*	—	75.2%*	13.8%*	2.1%	100%
Greptile	5.5%	4.4%*	71.3%**	18.0%*	—	100%
Sourcery	4.4%	—	85.6%	1.0%*	7.8%*	100%

*/**/* * indicates a significant difference from the overall AI Agents result reported in RQ1 with a small/medium/large effect size. In the column headers.

B.2 Interaction Patterns between human and AI agent reviewers in RQ2.

To understand whether the interaction patterns vary by the AI agent, we compare the rounds of back-and-forth discussion during review conversations initiated by each AI agent, and how often those conversations lead to a code change (Usefulness), against the overall AI agent. As shown in Table A3, when Claude Code initiates a review, conversations involve significantly (small effect sizes) less back-and-forth discussion and have lower usefulness than reviews initiated by AI agents overall. However, when Gemini Code Assist initiates a review, 19.8% of conversations involve further discussion, significantly higher than 14.6% overall. The remaining differences are not statistically significant.

TABLE A3: The interaction patterns comparison across each AI agent reviewer. AvgC is the average number of comments per conversation; >1C is the percentage of conversations with more than one comment, and the usefulness rate.

Agent	AvgC	>1C	Useful
All AI Agents	1.22	14.6%	31.2%
GitHub Copilot	1.18	17.2%	30.5%
CodeRabbit	1.26	11.2%	26.1%
Gemini Code Assist	1.22	19.8%*	29.2%
Cursor	1.23	20.9%	26.9%
Claude Code	1.08	7.3%*	22.5%*
Greptile	1.26	18.8%	33.9%
Sourcery AI	1.27	15.7%	26.6%

* indicates a significant difference from the All AI Agents result reported in RQ2 with a small effect size.

B.3 Code Metrics Changes after Suggestion Adoption in RQ3.

To assess whether changes in **code metrics** associated with adopted suggestions vary across AI agent reviewers, we calculate the change in 111 metrics for the source file before and after each suggestion is adopted (ΔM). As shown in Table A4, Greptile-generated adopted suggestions show a statistically significant decrease in the cyclomatic complexity of 0.979, compared to an increase of 0.045 for adopted suggestions generated by all AI agent reviewers. They also show decreases in the number of declarative statements and average lines of code that differ significantly from the increases observed for adopted AI agent suggestions overall. Moreover, adopted suggestions generated by Cursor and Sourcery AI also show decreases in the number of executable statements that differ significantly from the increase observed for adopted AI agent suggestions overall. The mean decreases are 0.825 and 1.271, respectively, compared with an overall increase of 0.093. The remaining agents (i.e., Copilot, CodeRabbit, Gemini Code Assist, and Claude Code) show no significant differences from the overall AI-agent distribution across the 111 code metrics.

TABLE A4: The code-quality impact comparison across each AI agent reviewer. Mean ΔM (Agent) is the mean metric delta for that agent’s adopted suggestions; Baseline (All) is the overall AI agent reviewer result in RQ3 III-C); “—” marks an agent with no significant metric.

Agent	Metric	Mean ΔM (Agent)	Baseline (All)
Copilot	—	—	—
CodeRabbit	—	—	—
Gemini	—	—	—
Cursor	Executable Statements	−0.825	0.093
Claude	—	—	—
Greptile	Declarative Statements	−0.729	0.025
	Sum Modified CC	−0.979	0.045
	Average Code Lines	−3.031	0.056
Sourcery	Executable Statements	−1.271	0.093

CC = Cyclomatic Complexity.